

Data Structures

CSC 301

Lecture Notes

Chapter 1: Data Types and Memory Representation

Computer Science
Osun State University
2026/2027

Prepared by
Tunde Sardine

Contents

1	Data Types and Memory Representation	2
1.1	Introduction	2
1.2	Primitive Data Types	2
1.2.1	Integer Types	2
1.2.2	Floating-Point Types	2
1.2.3	Character Types	3
1.2.4	Boolean Types	3
1.3	Memory Representation	3
1.3.1	Binary Representation	3
1.3.2	Memory Addressing	3
1.3.3	Endianness	3
1.3.4	Memory Alignment and Padding	4
1.4	Data Representation in Memory	4
1.4.1	Two's Complement Representation	4
1.4.2	IEEE 754 Floating-Point Representation	4
1.4.3	Character Encoding	4
1.5	Conclusion	5

Chapter 1

Data Types and Memory Representation

1.1 Introduction

Data structures are built upon the fundamental data types provided by a programming language and the way these types are represented in computer memory. Understanding primitive data types and their memory representation is essential for designing efficient data structures and algorithms. This chapter introduces the basic building blocks of data — primitive types — and examines how they are stored, addressed, and organized in memory.

1.2 Primitive Data Types

Primitive Data Types are the basic data types directly supported by a programming language. They represent simple values and are not composed of other types. Common categories include integers, floating-point numbers, characters, and booleans.

1.2.1 Integer Types

Integer types represent whole numbers. They vary in size (number of bits) and whether they are signed or unsigned. Typical sizes are 8, 16, 32, and 64 bits. Signed integers use the most significant bit as a sign bit; the most common representation for negative values is **Two's Complement** (see Section 1.4.1).

1.2.2 Floating-Point Types

Floating-point types represent real numbers with fractional parts. They follow the **IEEE 754** standard, which defines single precision (32 bits) and double precision (64 bits) formats. A floating-point number consists of a sign bit, an exponent, and a mantissa (significand).

1.2.3 Character Types

Character types store individual characters. Historically, a single byte (8 bits) was used with the ASCII encoding (0–127). Modern systems use Unicode, which may require multiple bytes per character (e.g., UTF-8, UTF-16).

1.2.4 Boolean Types

Boolean types represent logical values: `true` and `false`. They are typically stored as a single byte or even a single bit when packed into larger structures.

1.3 Memory Representation

Memory Representation refers to how data values are encoded as bits and organized in the addressable memory of a computer. Key concepts include binary encoding, memory addressing, endianness, and alignment.

1.3.1 Binary Representation

All data in a digital computer is ultimately represented as sequences of bits (0 and 1). The interpretation of a bit pattern depends on the data type. For example, the 8-bit pattern 01000001 can represent the integer 65, the character 'A' in ASCII, or a fragment of a larger value.

1.3.2 Memory Addressing

Memory is organized as a sequence of addressable units, typically bytes (8 bits). Each byte has a unique address. Multi-byte data types occupy consecutive addresses. The address of a variable is the address of its first byte.

1.3.3 Endianness

When a multi-byte value is stored in memory, the order of bytes matters. In **big-endian** systems, the most significant byte is stored at the lowest address. In **little-endian** systems, the least significant byte is stored at the lowest address. Most modern processors (x86, ARM) use little-endian, but network protocols often use big-endian (network byte order).

Example: 32-bit integer 0x12345678 stored at address 0x1000

Big-endian:

Address: 0x1000 0x1001 0x1002 0x1003

Byte: 0x12 0x34 0x56 0x78

Little-endian:

Address: 0x1000 0x1001 0x1002 0x1003

Byte: 0x78 0x56 0x34 0x12

1.3.4 Memory Alignment and Padding

Many architectures require that data of size n bytes be stored at addresses that are multiples of n (e.g., a 4-byte integer at an address divisible by 4). This is called **Memory Alignment**. To satisfy alignment, compilers may insert **Padding** bytes between structure fields or at the end of a structure. Padding increases memory usage but can improve access speed and prevent hardware exceptions.

1.4 Data Representation in Memory

This section details the specific bit-level representation of common primitive types.

1.4.1 Two's Complement Representation

Two's Complement is the standard method for representing signed integers. For an n -bit number, the range is $[-2^{n-1}, 2^{n-1} - 1]$. To negate a number, invert all bits and add 1. This representation allows addition and subtraction to be performed identically for signed and unsigned operands.

Example (8-bit):

- +5: 00000101
- -5: invert 00000101 $\rightarrow$ 11111010, add 1 $\rightarrow$ 11111011

1.4.2 IEEE 754 Floating-Point Representation

The **IEEE 754** standard defines:

- **Single precision (32 bits)**: 1 sign bit, 8 exponent bits (bias 127), 23 mantissa bits.
- **Double precision (64 bits)**: 1 sign bit, 11 exponent bits (bias 1023), 52 mantissa bits.

The value is $(-1)^{\text{sign}} \times 1.\text{mantissa} \times 2^{\text{exponent}-\text{bias}}$. Special values include zero, infinity, and NaN (Not a Number).

1.4.3 Character Encoding

- **ASCII**: 7-bit code (0–127) for English characters, often stored in 8 bits with the high bit zero.
- **UTF-8**: Variable-length encoding (1 to 4 bytes) for Unicode, backward compatible with ASCII.
- **UTF-16**: 2 or 4 bytes per character, used in some systems (e.g., Java, Windows).

1.5 Conclusion

Primitive data types and their memory representation form the foundation upon which all data structures are built. The choice of representation affects the range and precision of values, memory consumption, and performance. Understanding binary encoding, endianness, alignment, and standards like Two's Complement and IEEE 754 enables programmers to make informed decisions when designing data structures and to avoid subtle bugs related to data representation.

Tutorial Questions

1. List the four main categories of primitive data types and give a typical size (in bits) for each.
2. Explain the difference between signed and unsigned integer types. What is the range of an 8-bit signed integer in Two's Complement representation?
3. Describe the Three components of an IEEE 754 single-precision floating-point number. What is the bias for the exponent?
4. Compare big-endian and little-endian byte ordering. Show how the 32-bit hexadecimal value `0xDEADBEEF` would be stored in memory starting at address `0x2000` for each endianness.
5. What is memory alignment? Why might a compiler insert padding bytes into a structure?
6. Given the 8-bit Two's Complement representation `11110110`, what decimal value does it represent? Show your work.
7. A structure contains a `char` (1 byte), an `int` (4 bytes), and a `short` (2 bytes) in that order. Assuming 4-byte alignment for `int` and 2-byte alignment for `short`, draw the memory layout including any padding bytes. What is the total size of the structure?
8. Why is the Two's Complement representation preferred over sign-magnitude for signed integers in modern computers?